PROJECT PROPOSAL

ShelfSync — Digital Library Management System

Project Title	ShelfSync — Digital Library Management System
Project Type	Full-Stack Web Application
Technology Stack	Laravel 11, PHP 8.x, MySQL, Blade, Vanilla CSS, Gemini AI API
Submitted By	[Your Name / Group Names]
Student ID(s)	[Your Student IDs]
Department	[Your Department]
Institution	[Your University Name]
Course	Software Engineering / Web Application Development
Supervisor	[Supervisor Name]
Submission Date	April 2026

Table of Contents

1. Executive Summary
2. Problem Statement
3. Motivation & Rationale
4. Proposed Solution Overview
5. System Features & Capabilities
6. System Architecture
7. Technology Stack
8. Target Audience & Stakeholders
9. Project Objectives
10. Expected Outcomes & Benefits
11. Project Timeline
12. Conclusion

1. Executive Summary

ShelfSync is a next-generation, fully web-based Digital Library Management System designed to replace the paper-driven, inefficient library workflows that are commonplace in universities and colleges across Bangladesh and similar institutions globally. Built on the Laravel PHP framework with a modern glassmorphic user interface and powered by the Google Gemini 2.5 Flash artificial intelligence API, ShelfSync automates the complete lifecycle of library operations — from student registration and digital ID card issuance to book rental management, overdue fine calculation, and intelligent book discovery.

The system serves three distinct user groups simultaneously: students who need a fast, mobile-friendly way to browse and rent books; librarians and administrators who need powerful management tools; and unauthenticated guests who can still explore the catalog and interact with the AI book assistant without creating an account. ShelfSync is not merely a CRUD application — it is a production-grade software system with real-time availability tracking, a rule-based AI assistant, an automated fine dispute resolution workflow, and a premium design language that reflects the standards expected of modern web software.

2. Problem Statement

Library management in most educational institutions is still handled through a combination of physical ledgers, handwritten forms, and outdated desktop software. This approach introduces a range of operational, administrative, and user experience problems:

2.1 Physical Library Cards

Students are issued physical library cards that are frequently lost, damaged, or forgotten. Replacing a card is a manual process involving paperwork, photographs, and administrative approval. There is no instant verification mechanism — a librarian cannot digitally confirm a student's identity or membership status without physically inspecting the card.

2.2 Manual Book Tracking

Inventory management relies on manual entries in registers or spreadsheets. When a book is borrowed, a librarian manually records it. When it is returned, another entry is made. This process is prone to human error, duplication, and loss of data. There is no real-time visibility into how many copies of a book are available at any given moment.

2.3 No Accountability for Fines

Overdue fine calculation is inconsistent. Librarians must manually check return dates, calculate overdue periods, and inform students. Students often dispute fines without a formal mechanism, and there is no audit trail to resolve disagreements.

2.4 No Digital Discovery Tool

Students cannot search the catalog remotely, check availability before visiting the library, or get information about a book's contents before deciding to rent it. This results in wasted trips and under-utilization of the library's collections.

2.5 Administrative Bottlenecks

Administrators have no consolidated dashboard. Approving rental requests, managing returns, handling appeals, and generating reports all require separate tools or manual processes, leading to delays and frustration for both staff and students.

2.6 Absence of Intelligent Assistance

No existing library system used in typical university settings provides AI-powered book summaries or Q&A, leaving students without a guided discovery experience.

---

3. Motivation & Rationale

The motivation for ShelfSync arises directly from the observation of these pain points in real academic environments. A digitized, automated, and aesthetically refined library system can:

- Save hours of administrative time per week
- Eliminate fine disputes through transparent, automated tracking
- Give students 24/7 access to library services from any device
- Reduce paper waste and physical storage overhead
- Create a measurable audit trail for all transactions
- Provide AI-guided discovery that encourages more students to read

ShelfSync was also motivated by the gap in the local software landscape — existing library management systems (such as Koha or OpenBiblio) are complex, desktop-oriented, and require significant technical expertise to deploy and maintain. ShelfSync is a clean, cloud-ready, Laravel-based system that can be deployed on shared hosting with minimal configuration.

---

4. Proposed Solution Overview

ShelfSync proposes a unified web application that handles every aspect of library management through a browser. The application is structured around the **Model-View-Controller (MVC)** architectural pattern, ensuring clear separation of concerns and long-term maintainability.

At its core, ShelfSync provides:

- A **public-facing landing page** with live search, book catalog, and AI assistant access — accessible to anyone without login
- A **student portal** where authenticated users can rent books, manage their digital library card, track reading progress, manage wishlists, and appeal fines
- An **admin panel** where librarians can manage inventory, approve or reject all student actions, handle fines, and view system-wide activity
- A **Gemini AI integration** that provides intelligent, book-specific information and conversation to any user, powered by Google's Gemini 2.5 Flash model

All user interactions are secured with CSRF protection, role-based middleware, and server-side validation. The AI API key is never exposed to the client.

---

5. System Features & Capabilities

5.1 Public Features (No Login Required)

- **Landing Page & Book Discovery:** A visually rich homepage showcasing the system's value proposition, available books, features, and a FAQ section.
- **Live Book Search:** A real-time search bar that queries the database as the user types, returning matching results by title, author, and category.
- **Book Catalog Browsing:** A filterable, paginated grid of all books with cover images, category, author, price, and availability count.
- **AI Book Assistant:** By clicking an info icon (ⓘ) on any book card, any user (including guests) can open a modal powered by Gemini 2.5 Flash that generates structured metadata and enables a restricted Q&A chat about the specific book.
- **Contact Form:** A public form for submitting inquiries, stored in the database.

5.2 Student / User Features

- **Registration, Login, Email Verification:** Standard authentication flow via Laravel Breeze.
- **Profile Management:** Students can update their name, email, address, and password.
- **Book Rental Submission:** Authenticated students can request to rent any available book by specifying a return date.
- **My Rentals Dashboard:** A personal view of all active, past, approved, and rejected rentals with fine information.
- **Digital Library Card Application:** Students apply for a digital library card; upon admin approval, a glassmorphic digital identity card is generated with 6-month validity.
- **Card Renewal:** Students can request renewal of an expired card.
- **Reading Progress Tracker:** Students can log books they are actively reading, update their progress, and mark books as completed.
- **Wishlist:** An AJAX-powered toggle that allows students to bookmark books for future renting.
- **Fine Appeal Submission:** Students can submit a written appeal for any disputed fine.

5.3 Admin / Librarian Features

- **Admin Dashboard:** Consolidated overview of users, books, active rentals, pending requests, and outstanding fines.
- **Rental Approval Workflow:** Admins approve or reject rental requests; mark approved rentals as returned.
- **Returns Scanner:** Dedicated interface for processing returns by rental ID.

- **Book Management (CRUD):** Full create, read, update, delete for books with image URLs and metadata.
- **Category Management:** Add or delete book categories.
- **Library Card Management:** Approve, reject, or revoke digital library cards.
- **Fine Management:** View, clear, or adjust outstanding fines.
- **Fine Appeal Resolution:** Review and resolve or reject student fine appeals.
- **Member Management:** View all users, promote to admin, or demote from admin.

---

6. System Architecture

ShelfSync follows the **MVC (Model-View-Controller)** pattern as enforced by the Laravel framework.

```
┌──────────────────────────────────────────────────────────────────────┐
│                          BROWSER (Client)                            │
│     Blade Views + Vanilla CSS + JavaScript (AJAX for AI & Search)    │
└──────────────────────────────────────────────────────────────────────┘
                            │
                            │  HTTP Request
                            ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      ROUTER (routes/web.php)                         │
│             Maps URLs to Controller methods                          │
│     Applies Middleware: auth | admin | throttle | CSRF               │
└──────────────────────────────────────────────────────────────────────┘
                            │
                            │
          ┌─────────────────┼──────────────────┐
          ▼                 ▼                  ▼
  UserController    AdminController       AiController
  (Student logic)   (Admin logic)         (Gemini AI)
          │                 │                  │
          └─────────────────┼──────────────────┘
                            ▼
┌──────────────────────────────────────────────────────────────────────┐
│                     MODELS (Eloquent ORM)                            │
│  User · Book · Category · Rental · LibraryCard · FineAppeal         │
│  Wishlist · ReadingProgress · ChatHistory · Contact                  │
└──────────────────────────────────────────────────────────────────────┘
                            │
                            ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        MySQL DATABASE                                │
│  10 tables · Relational integrity via foreign keys                   │
└──────────────────────────────────────────────────────────────────────┘
                                          │
                                          ▼
                              ┌──────────────────────────┐
                              │   Google Gemini AI API   │
                              │  (v1beta · gemini-2.5-flash) │
                              │   Server-side calls only │
                              └──────────────────────────┘
```

Key Architectural Decisions:

- The Gemini API key is stored in `.env` and never sent to the browser.
- All AI endpoints are rate-limited to 15 requests per minute per IP.
- Blade components are reused across user and admin layouts.
- Role-based access control is implemented through a custom `admin` middleware checking the `role` field on the authenticated user.

---

7. Technology Stack

Layer	Technology	Justification
Backend Framework	Laravel 11 (PHP 8.x)	Mature MVC framework with built-in auth, ORM, middleware, and routing.
Database	MySQL (via XAMPP)	Reliable relational database; well-supported by Laravel's Eloquent ORM.
ORM	Eloquent	Relationships (`hasMany` , `belongsTo`) cleanly map to the database schema.
Authentication	Laravel Breeze	Lightweight scaffolding for registration, login, email verification.
Frontend	Blade Templating + Vanilla CSS	Custom CSS design system with `--ss-` tokens. No JavaScript framework overhead.
AI Integration	Google Gemini 2.5 Flash (REST API v1beta)	Capable of structured JSON output, multi-turn conversation, and system instruction enforcement.
UI Design Language	Glassmorphism, Dark Mode	`backdrop-filter: blur()` , gradient overlays, CSS animations for a premium aesthetic.
Web Server	Apache (XAMPP)	Standard PHP hosting environment.

---

8. Target Audience & Stakeholders

Stakeholder	Role in the System	Key Needs
Students	Primary users who rent books and manage their library membership	Easy browsing, fast rental, transparent fines, mobile access

Librarians / Admins	Operators managing inventory and approving requests	Efficient workflows, consolidated dashboard, audit trail
University Institution	Owner of the system	Reduced paper costs, accountability, scalability
Guests / Visitors	Prospective students or general public	Catalog access and AI book discovery without registration

---

9. Project Objectives

1. Digitize the complete library membership lifecycle (registration → card issuance → expiry → renewal → revocation).
2. Automate book rental approval, return processing, and fine calculation.
3. Eliminate physical library cards with a database-backed digital identity card.
4. Enable real-time book availability so students always see accurate stock counts.
5. Integrate an AI-powered book assistant accessible to all users including guests.
6. Provide a transparent fine management and dispute resolution system.
7. Deliver a premium, responsive user interface reflecting professional web standards.
8. Enforce security best practices: CSRF protection, role-based access control, server-side API key management, and rate limiting.

---

10. Expected Outcomes & Benefits

Benefit	Details
Time Savings	Admin approval workflows reduce per-transaction time from 10+ minutes (manual) to under 30 seconds.
Accuracy	Automated fine calculation eliminates arithmetic errors; inventory is always accurate.
Accessibility	24/7 access from any device — no need to physically visit the library to check availability.
Transparency	Students can see their fine history, appeal status, and card status in real time.
AI Discovery	Students are guided to explore new books through an AI assistant that explains themes and answers questions.
Paperless Operation	All forms, cards, and records are digital, reducing physical storage and paper costs.

Scalability	The system can be extended to support multiple branches, barcode scanners, or email notifications with minimal changes.

---

11. Project Timeline

Phase	Activities	Duration
Phase 1 — Planning	Requirements gathering, database design, wireframing	Week 1–2
Phase 2 — Database & Models	Migrations, Eloquent models, relationships	Week 3
Phase 3 — Auth & Roles	Breeze setup, middleware, role management	Week 4
Phase 4 — Core Features	Rental system, fines, library card, wishlist, progress tracker	Week 5–7
Phase 5 — Admin Panel	Dashboard, all CRUD operations, approval workflows	Week 8–9
Phase 6 — AI Integration	Gemini API, modal UI, chat persistence	Week 10
Phase 7 — UI Polish	Design system, glassmorphism, animations, responsive design	Week 11
Phase 8 — Testing & Deployment	Bug fixing, edge cases, final deployment	Week 12

---

12. Conclusion

ShelfSync represents a significant step forward from the manual, paper-based library systems currently in use at many academic institutions. By combining a robust Laravel backend, a thoughtfully designed relational database, and the intelligence of Google's Gemini AI, the project delivers a solution that is not only functionally complete but also aesthetically modern and architecturally sound.

Every feature — from the glassmorphic digital library card to the AI-powered book assistant — was designed with real user needs in mind. The project demonstrates mastery of full-stack web development, database design, API integration, security practices, and UI/UX principles. ShelfSync is ready for real-world deployment and can serve as a foundation for future enhancements including mobile applications, hardware scanner integration, and multi-branch support.

Mahadi (232-35-040)